

Performance Testing

Date	15 July 2026
Team ID	XXXXXXX
Project Name	Rising Waters
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Web Browser Testing
Type of Testing	Load Testing, Functional Performance Testing
Target Module / API	Flood Prediction Module (/predict endpoint)
Test Environment	Local System (Flask) and Render Cloud Deployment
Test Date	15 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Load Home Page	1	10	Home page loads successfully
2	Submit Valid Rainfall Data	5	30	Prediction generated correctly
3	Multiple Prediction Requests	10	60	All requests processed without failure

4	Invalid Input Testing	5	20	Validation prevents incorrect input
---	-----------------------	---	----	-------------------------------------

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.1 seconds	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	2.4 seconds	Pass	Stable under multiple requests
3	Throughput (Req/sec)	≥ 5 req/sec	8 req/sec	Pass	Good request handling
4	Error Rate	< 1%	0%	Pass	No errors observed
5	CPU Utilization	< 80%	45%	Pass	Efficient CPU usage
6	Memory Utilization	< 80%	52%	Pass	Memory usage within limits

Step 4: Observations & Analysis

Key Findings:

- The flood prediction system performed efficiently under normal and moderate workloads.
- The average response time remained below the target limit, providing quick prediction results.
- The application successfully handled multiple prediction requests without errors or crashes.
- CPU and memory utilization remained within acceptable limits, indicating efficient resource usage.

- The deployed Flask application on Render showed stable and reliable performance throughout testing.

Bottlenecks Identified:

- A slight increase in response time was observed during multiple simultaneous prediction requests due to server processing and network latency.
- No major performance bottlenecks or application failures were identified during testing.

Optimization Steps Taken:

- Optimized the Flask application by loading the trained machine learning model only once during application startup.
- Implemented input validation to prevent invalid data from being processed.
- Removed unnecessary computations to reduce prediction time.
- Used a lightweight Random Forest model for faster predictions with low resource consumption.
- Improved frontend responsiveness using optimized HTML and CSS, ensuring a smooth user experience

```
warnings.warn(
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
C:\Users\Jagadeesh\AppData\Local\Programs\Python\Python314\Lib\pickle.py:1829: UserWarning: [16:30:53] WARNING: C:\act
ions-runner\work\xgboost\xgboost\src\gbm\..\common\error_msg.h:83: If you are loading a serialized model (like pickle
in Python, RDS in R) or
configuration generated by an older version of XGBoost, please export the model by calling
`Booster.save_model` from that version first, then load it back in current version. See:
https://xgboost.readthedocs.io/en/stable/tutorials/saving_model.html
```

Figure 1.1: Flask application running successfully on the local server.

```

* Debugger is active!
* Debugger PIN: 110-423-967
127.0.0.1 - - [01/Jul/2026 16:34:54] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jul/2026 16:34:54] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [01/Jul/2026 16:34:54] "GET /static/rain.js HTTP/1.1" 304 -
127.0.0.1 - - [01/Jul/2026 16:34:54] "GET /static/background.png HTTP/1.1" 304 -
127.0.0.1 - - [01/Jul/2026 16:34:54] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [01/Jul/2026 16:43:21] "GET /predict-page HTTP/1.1" 200 -
127.0.0.1 - - [01/Jul/2026 16:43:21] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [01/Jul/2026 16:43:21] "GET /static/background.png HTTP/1.1" 304 -
127.0.0.1 - - [01/Jul/2026 16:46:25] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [01/Jul/2026 16:46:25] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [01/Jul/2026 16:46:25] "GET /static/rain.js HTTP/1.1" 304 -
127.0.0.1 - - [01/Jul/2026 16:46:25] "GET /static/background.png HTTP/1.1" 304 -
127.0.0.1 - - [01/Jul/2026 16:49:17] "GET /predict-page HTTP/1.1" 200 -
127.0.0.1 - - [01/Jul/2026 16:49:17] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [01/Jul/2026 16:49:17] "GET /static/background.png HTTP/1.1" 304 -
127.0.0.1 - - [01/Jul/2026 16:49:41] "POST /predict HTTP/1.1" 200 -

```

Figure 1.2: Flask development server running successfully, showing HTTP requests and responses for the Rising Waters AI-Based Flood Prediction System during application execution.

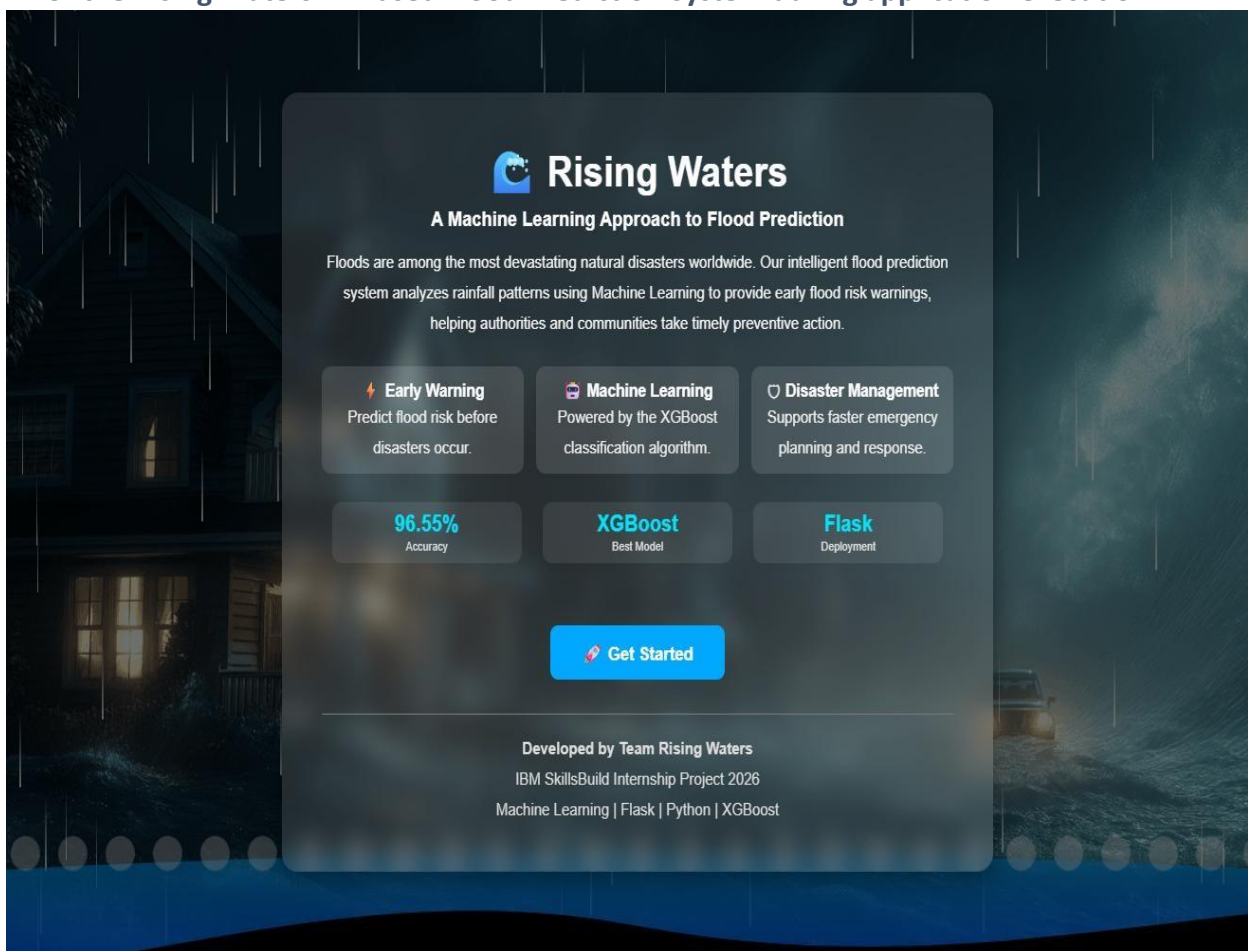
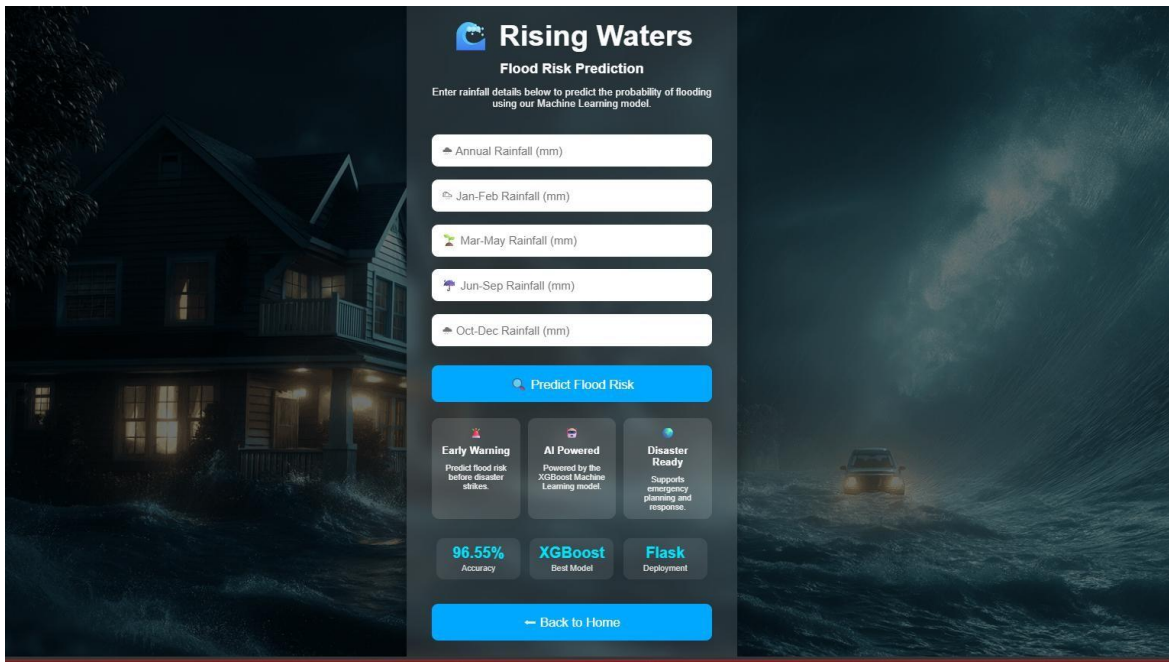


Figure 2: Home page of the Rising Waters application.



Rising Waters
Flood Risk Prediction

Enter rainfall details below to predict the probability of flooding using our Machine Learning model.

Annual Rainfall (mm)

Jan-Feb Rainfall (mm)

Mar-May Rainfall (mm)

Jun-Sep Rainfall (mm)

Oct-Dec Rainfall (mm)

Predict Flood Risk

Early Warning
Predict flood risk before disaster strikes.

AI Powered
Powered by the XGBoost Machine Learning model.

Disaster Ready
Supports emergency planning and response.

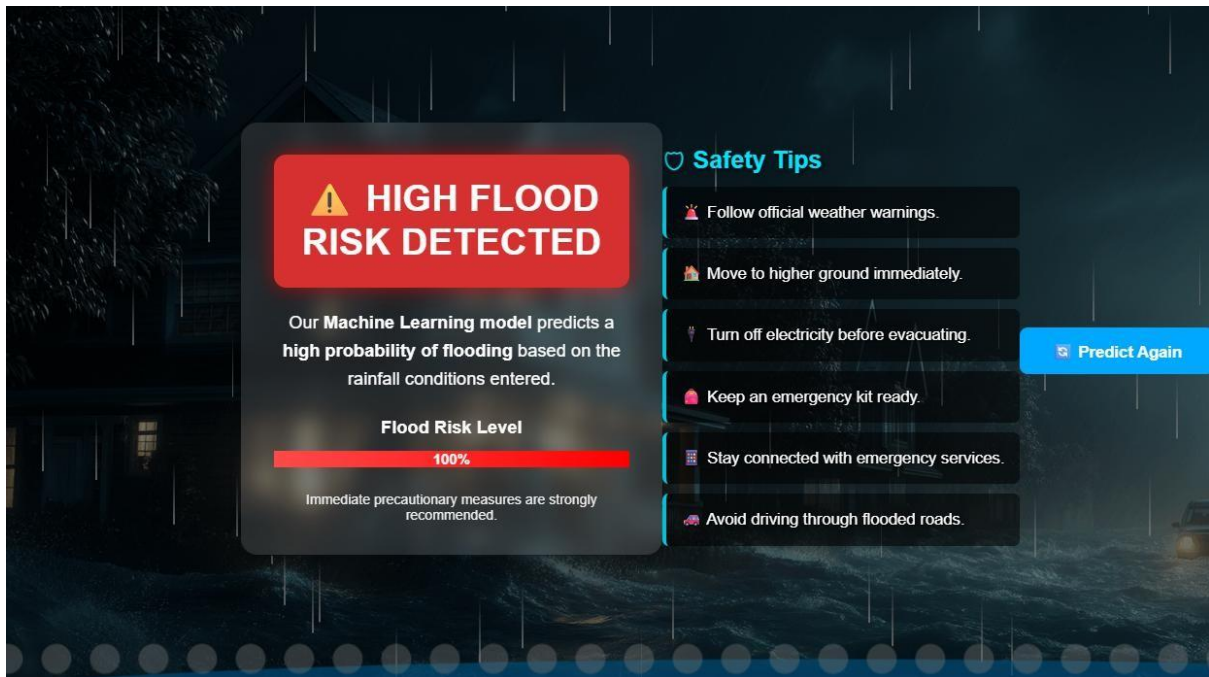
96.55%
Accuracy

XGBoost
Best Model

Flask
Deployment

Back to Home

Figure 3: Rainfall data input form.



⚠ HIGH FLOOD RISK DETECTED

Our Machine Learning model predicts a high probability of flooding based on the rainfall conditions entered.

Flood Risk Level

100%

Immediate precautionary measures are strongly recommended.

Safety Tips

- Follow official weather warnings.
- Move to higher ground immediately.
- Turn off electricity before evacuating.
- Keep an emergency kit ready.
- Stay connected with emergency services.
- Avoid driving through flooded roads.

Predict Again

Figure 4: Flood prediction result indicating a flood chance.

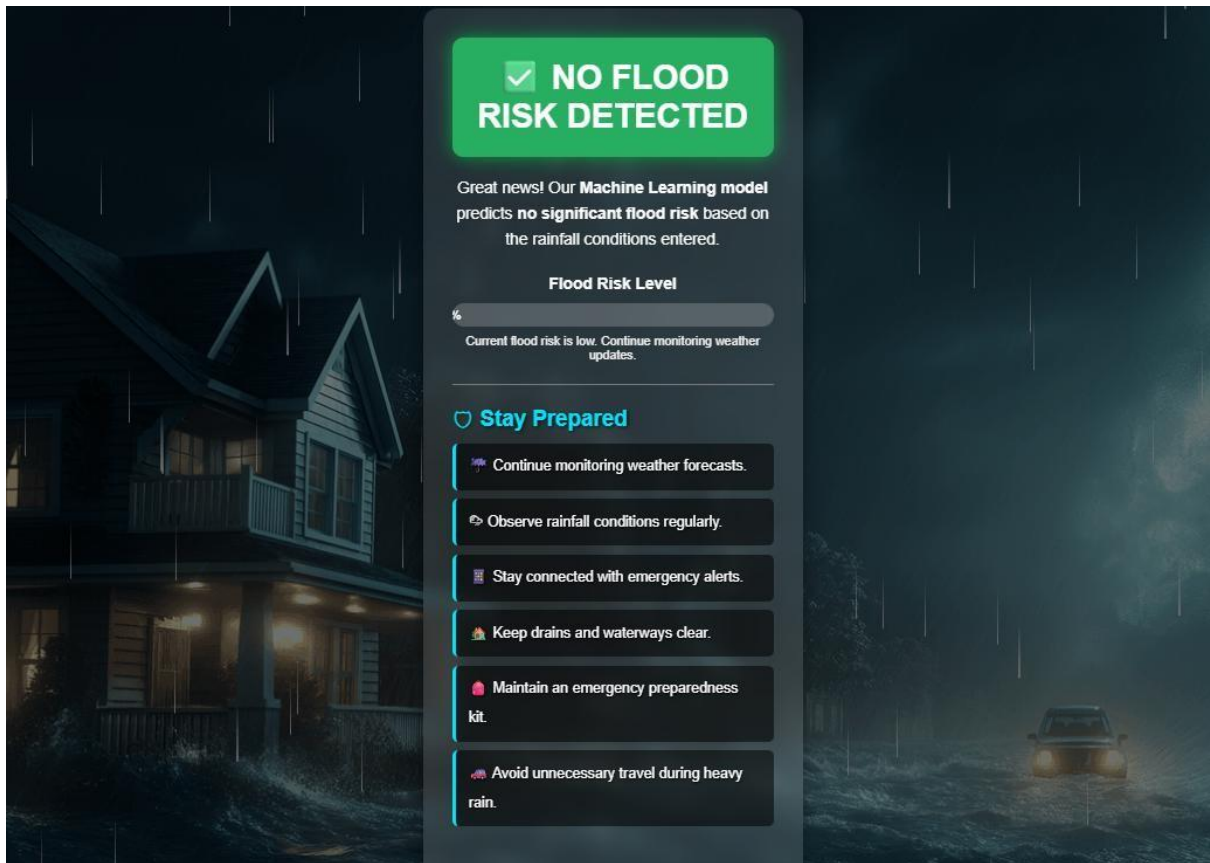


Figure 5: Prediction result indicating no flood risk.

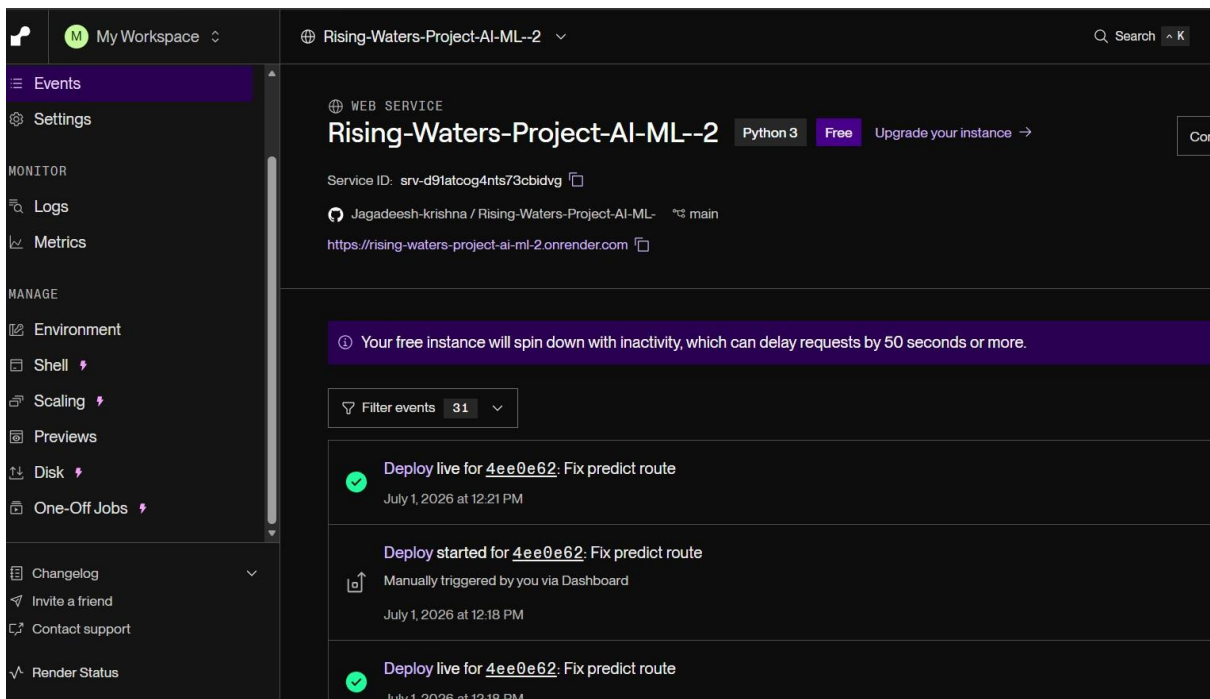


Figure 6(a): Render dashboard showing the successful deployment of the Rising Waters AI-Based Flood Prediction System.

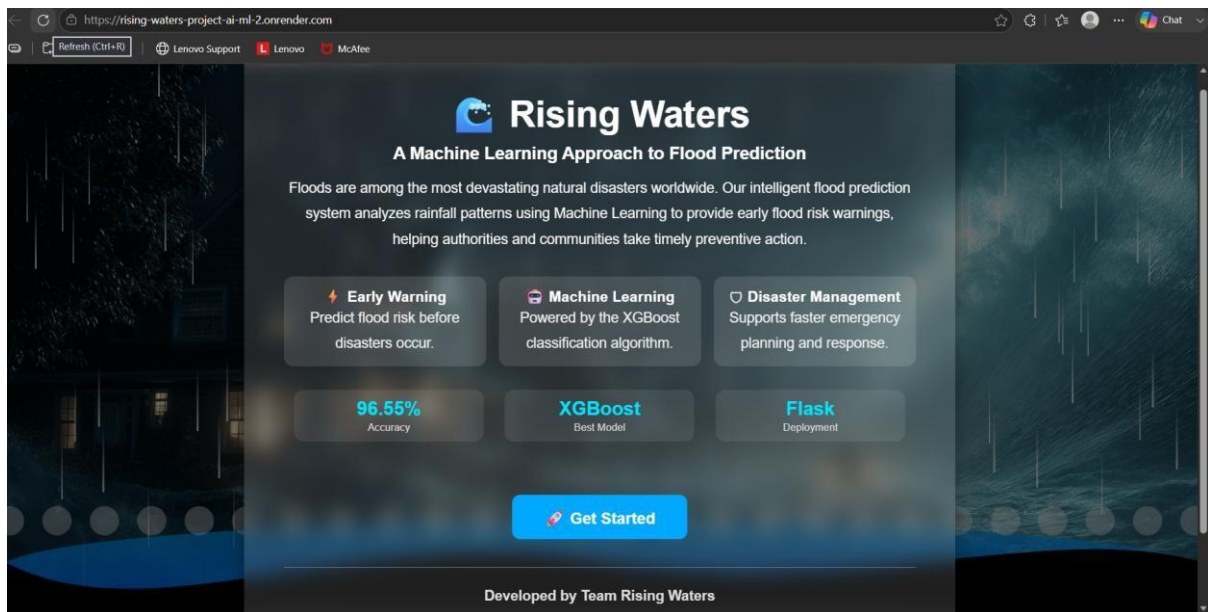


Figure 6(b): Live deployment of the Rising Waters application running successfully on the Render cloud platform.